

Sindh Madressatul Islam University Karachi

Object Oriented Programming



SPRING 2026

Lecture 1 of 16

Engr. Majid Kaleem

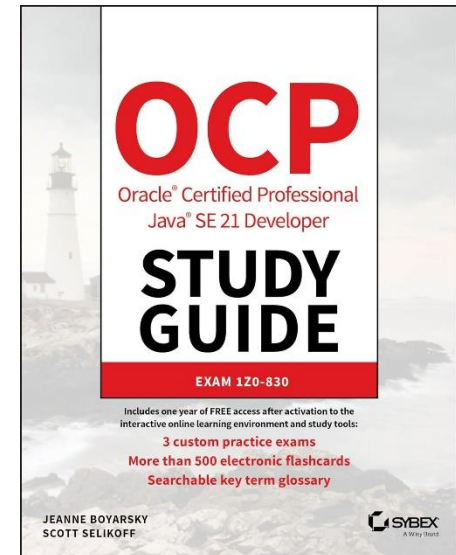
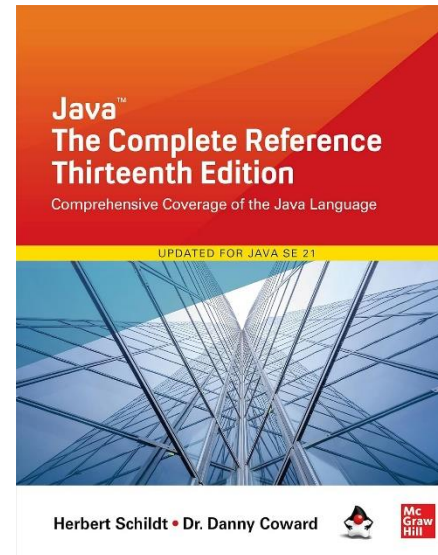
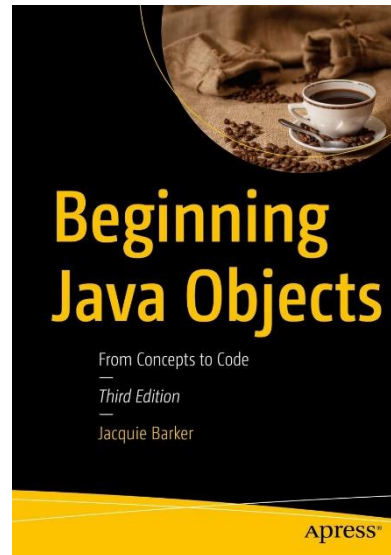
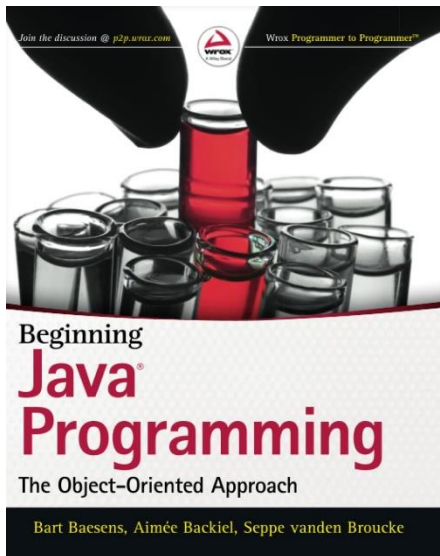
WEEKLY AGENDA

TENTATIVE WEEKLY DATES		TENTATIVE TOPICS
1	16 th Feb – 20 th Feb	INTRO. TO OBJECT-ORIENTED PROGRAMMING, PROCEDURAL VS.OOP, OOP PRINCIPLES, ETC.
2	23 rd Feb – 27 th Feb	CLASSES & OBJECTS
3	02 nd Mar – 06 th Mar	METHODS & MODULAR PROGRAMMING
4	09 th Mar – 13 th Mar	JAVA FUNDAMENTALS ASSIGNMENT #1 & QUIZ #1
5	16 th Mar – 20 th Mar	INHERITANCE
6	23 rd Mar – 27 th Mar	POLYMORPHISM
7	30 th Mar – 03 rd Apr	INTERFACES ASSIGNMENT #2 & QUIZ #2
8	06 th Apr – 10 th Apr	EXCEPTION HANDLING
← MID TERM EXAMINATIONS (13th APR 26 – 17th APR 2026) →		
9	20 th Apr – 24 th Apr	COLLECTIONS FRAMEWORK
10	27 th Apr – 01 st May	FILE HANDLING
11	04 th May – 08 th May	GUI PROGRAMMING
12	11 th May – 15 th May	APPLICATION DEVELOPMENT ASSIGNMENT #3 & QUIZ #3
13	18 th May – 22 nd May	DESIGN PRINCIPLES & PATTERNS
14	25 th May – 29 th May	ADVANCED OOP CONCEPTS
15	01 st Jun – 5 th Jun	CASE STUDIES AND INDUSTRY PRACTICES ASSIGNMENT #4 & QUIZ #4
16	08 th Jun – 12 th Jun	PRESENTATIONS & REVISION WEEK
← FINAL TERM EXAMINATIONS (15th JUN 26 – 19th JUN 2026) →		

Note

- This schedule is subject to revisions as conditions may warrant.*
- Topics will be covered in sequence no matter if the university observes any planned or unplanned holidays.*
- The information in this course outline is subject to revision as conditions may warrant.*

RECOMMENDED BOOKS



EXAM FORMAT & COVERAGE

IMPORTANT

MID TERM	FINAL TERM
The total number of questions will be four (04).	The total number of questions will be four (04).
Each question will comprise two parts; i.e., (a) & (b)	Each question will comprise two parts; i.e., (a) & (b)
Two questions will be from lecture slides, one from assignments, and one from classroom discussions and/or “self-study” topics.	Two questions will be from lecture slides, one from assignments, and one from classroom discussions and/or “self-study” topics.

WHAT IS OBJECT-ORIENTED PROGRAMMING?



- This course is not about Java!
- But to practice OOP concepts, we'll use Java.



ONLINE COMPILERS

- <https://onecompiler.com/java>

WHAT IS OBJECT-ORIENTED PROGRAMMING?

Type	Focus	Example
Procedural	Focuses on step-by-step instructions and structured procedures/functions to solve problems.	C, Pascal, Fortran, Etc.
Object-Oriented (OOP)	Focuses on modeling real-world entities using classes and objects.	Java, C++, C#, Etc.
Functional	Focuses on mathematical functions and transformations, avoiding mutable state and side effects.	Haskell, Lisp, Scala, Etc.
Scripting	Focuses on automation, rapid prototyping, and glue code for connecting systems or building quick applications.	Python, JavaScript, PHP, Ruby, Etc.
Logic	Focuses on rules, facts, and inference ; solving problems through logical reasoning rather than explicit instructions.	Prolog, Etc.
Machine/Low-level	Focuses on direct hardware control and efficiency , working close to the CPU and memory.	Assembly, Machine Code, Etc.

PROCEDURAL VS. OBJECT-ORIENTED PROGRAMMING

Paradigm	Focus	Example Question
Procedural Programming	Focuses on How (sequence of steps, procedures)	" How do we calculate student grades?"
Object-Oriented Programming	Focuses on: • Who (in some particular role). • Objects/entities and their interactions	" Who is the student, and what can they do ?"

EXAMPLE

Procedural Programming	Object-Oriented Programming
<pre>public fee() {...}; public salary() {...}; void main() { int stdId; int empld; String prog; String dept; }</pre>	<pre>class Student { int stdId; String prog; public fee() {...}; } class Faculty { int empld; String dept; public salary() {...}; }</pre>

WHAT IS OBJECT-ORIENTED PROGRAMMING?

- Object-Oriented Programming's main focus is objects (the "**Who**").
- Once the **Who** is identified, their **attributes** (**state**) and **behaviors** (methods) can easily be identified.
- This makes systems more modular, reusable, and closer to **real-world** thinking.
- **State**:
 - The state represents the values of all the attributes of an object at a **given time**. E.g., today your **age = 17**;
 - The state evolves with time. E.g., Next year **age = 18**;
 - Some state values may be constant. your **CNIC = 254173649097-1**;

WHAT IS OBJECT-ORIENTED PROGRAMMING?

- In OOP, we model (*represent*) the world by identifying **Who** the object is — once we know that, we can define **What** it has and **What** it does.
- An object is defined by **what it has** (**attributes**), **what it does** (**behavior**), and **who it is** (**identity**).

WHAT IS A CLASS?

- A class is like a *blueprint* or *template*.
- It doesn't do anything on its own — it just describes *what something should look like*.
- Class defines a *user-defined* data type.
- Its fields give it state, and its methods provide it with behavior.
- A class is a description or specification of a real-world entity, modeled from a *particular perspective* (*role*) relevant to the system.

WHAT IS A CLASS?

- “**Who**” can exist in multiple “**roles**” (perspectives):

// Lawyer class

```
class Lawyer {  
    // Attributes  
    private String name;  
    private String specialization;  
    private int yearsOfExperience;  
}
```

// Methods (behaviors)

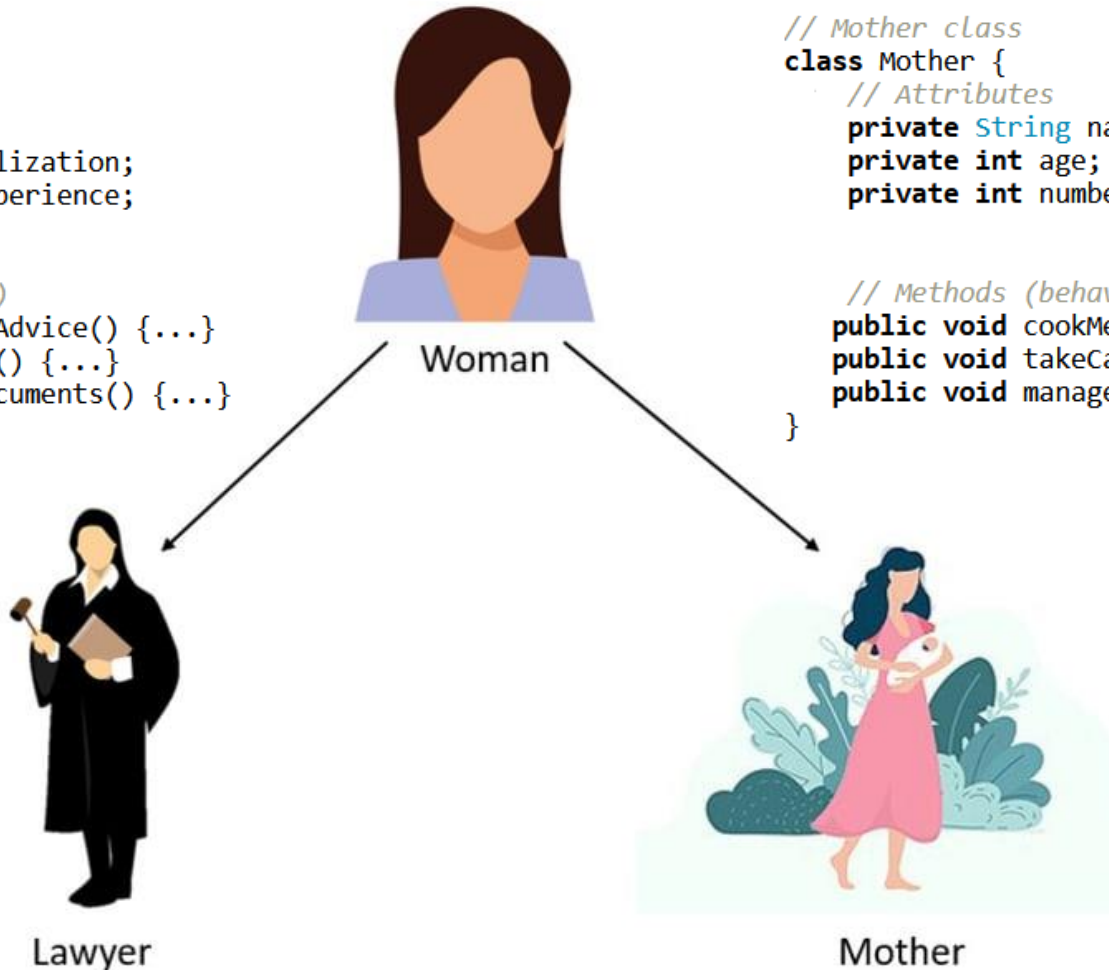
```
public void giveLegalAdvice() {...}  
public void argueCase() {...}  
public void prepareDocuments() {...}
```

// Mother class

```
class Mother {  
    // Attributes  
    private String name;  
    private int age;  
    private int numberOfChildren;  
}
```

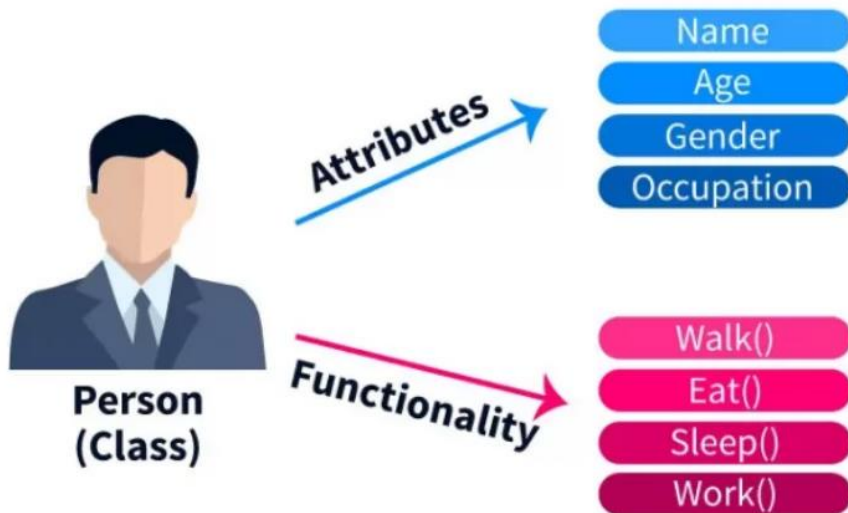
// Methods (behaviors)

```
public void cookMeal() {...}  
public void takeCareOfChildren() {...}  
public void manageHousehold() {...}
```



WHAT IS A CLASS?

- An object (instance) is the product.
- Object is a **variable** (instance) of type **Class**.



// Define the Person class

```
class Person {  
    // Attributes (Fields)  
    String name;  
    int age;  
    String gender;  
    String occupation;  
}
```

// Methods (Behaviors)

```
void walk() {...}  
void eat() {...}  
void sleep() {...}  
void work() {...}
```

```
}
```

// Main class to use Person

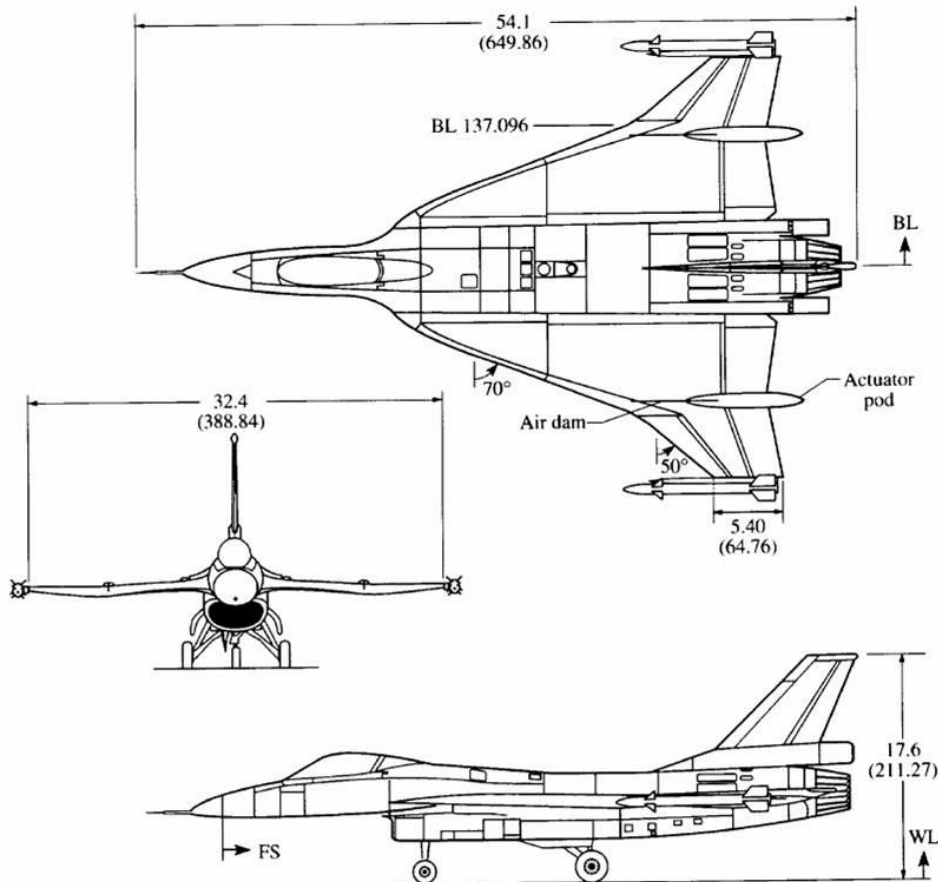
```
public class Main {  
    public static void main(String[] args) {  
        // Create two Person objects  
        Person ObjMajid = new Person();  
        Person ObjAdnan = new Person();  
    }  
}
```

CLASS VS. OBJECTS

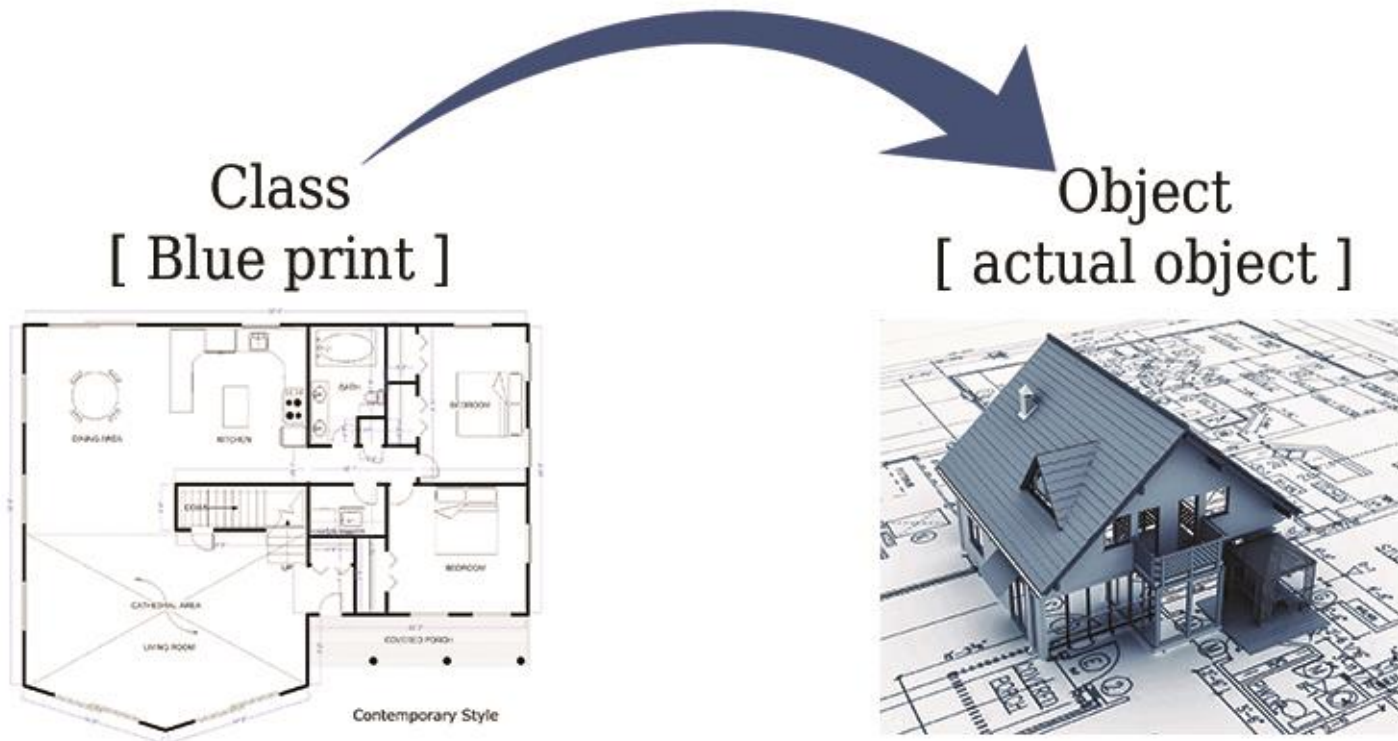
EXAMPLE

Class
[Blue print]

Object
[actual object]

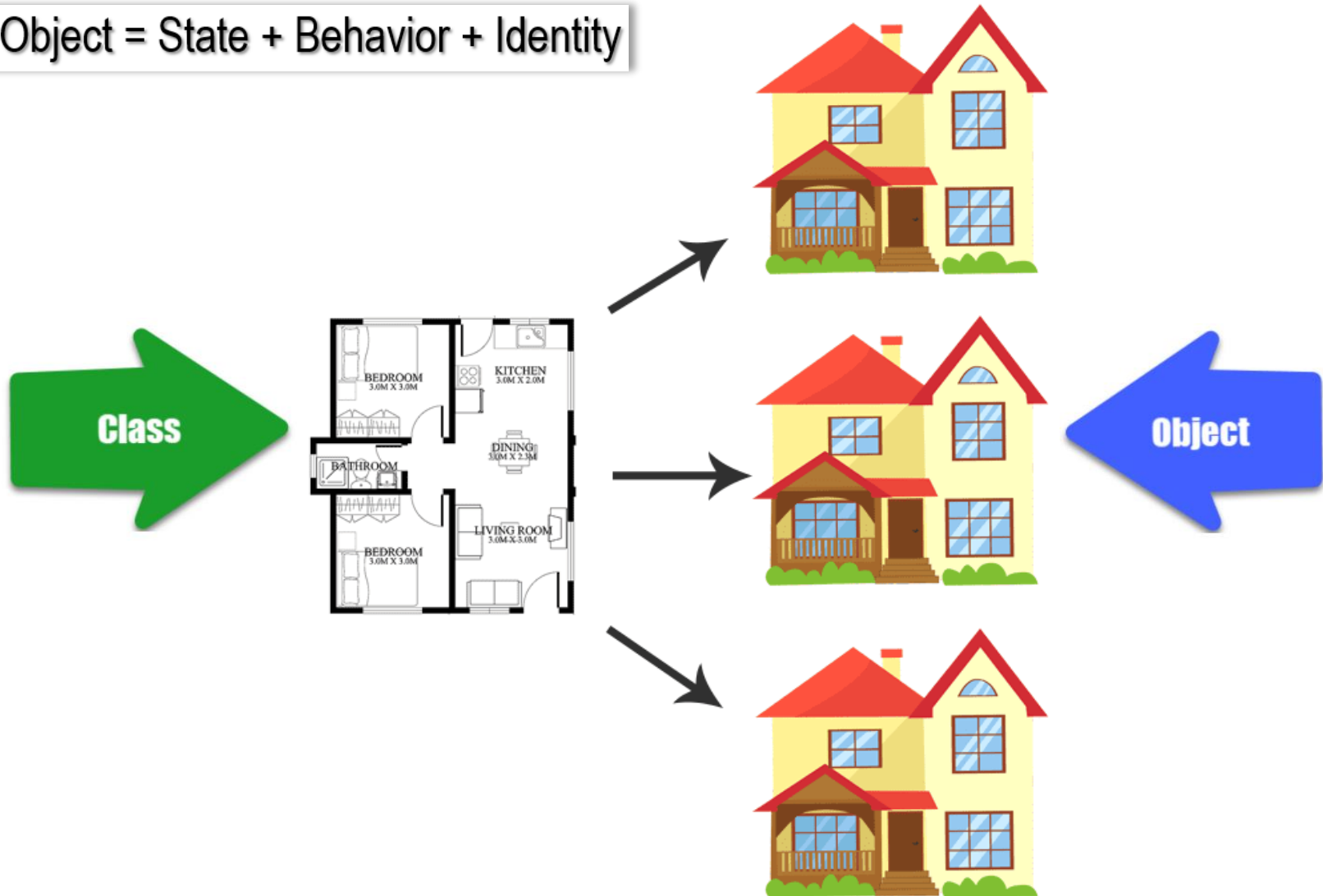


- **Class** is used to create **Objects**.



- **Object** is a variable (*instance*) of type **Class**.

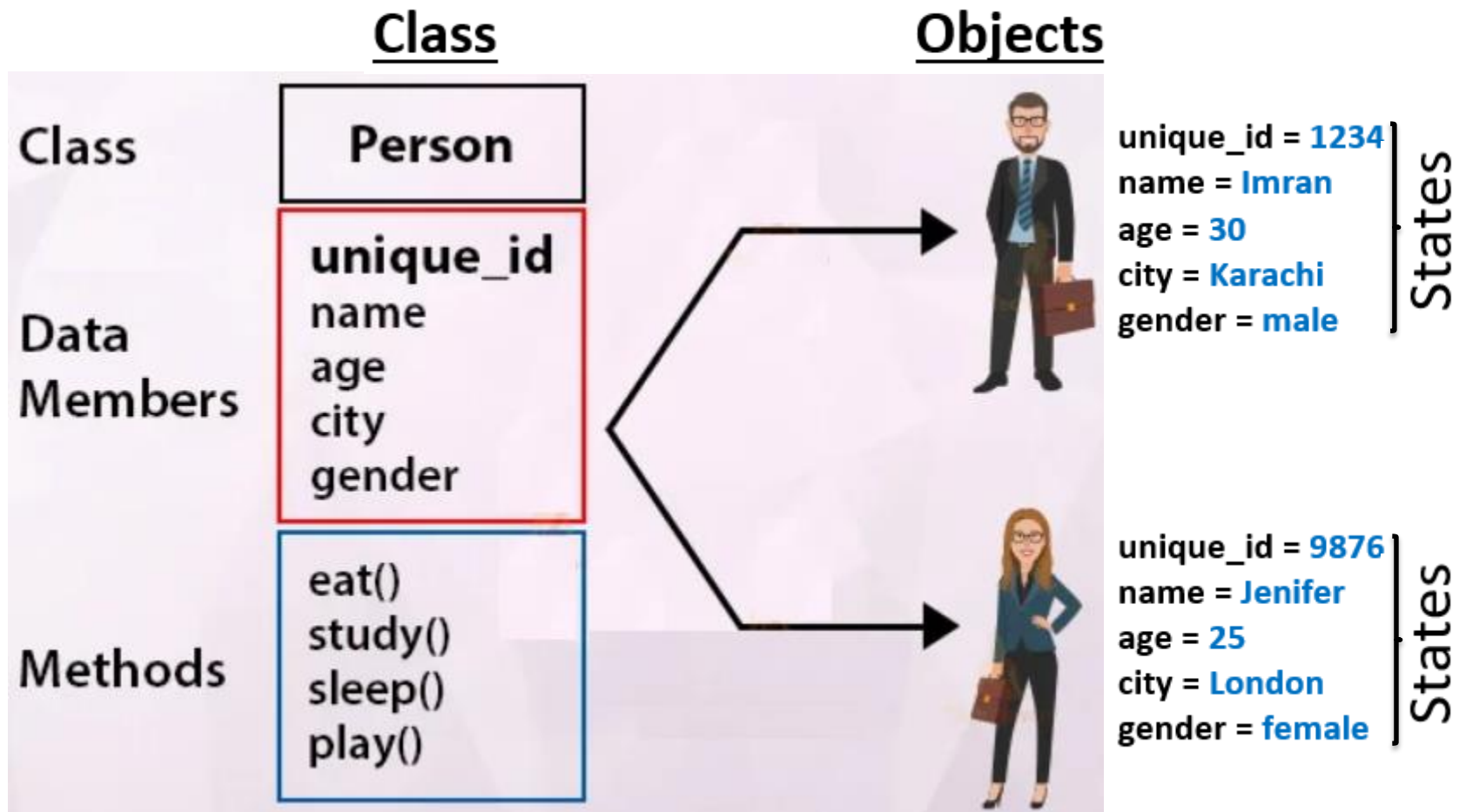
Object = State + Behavior + Identity



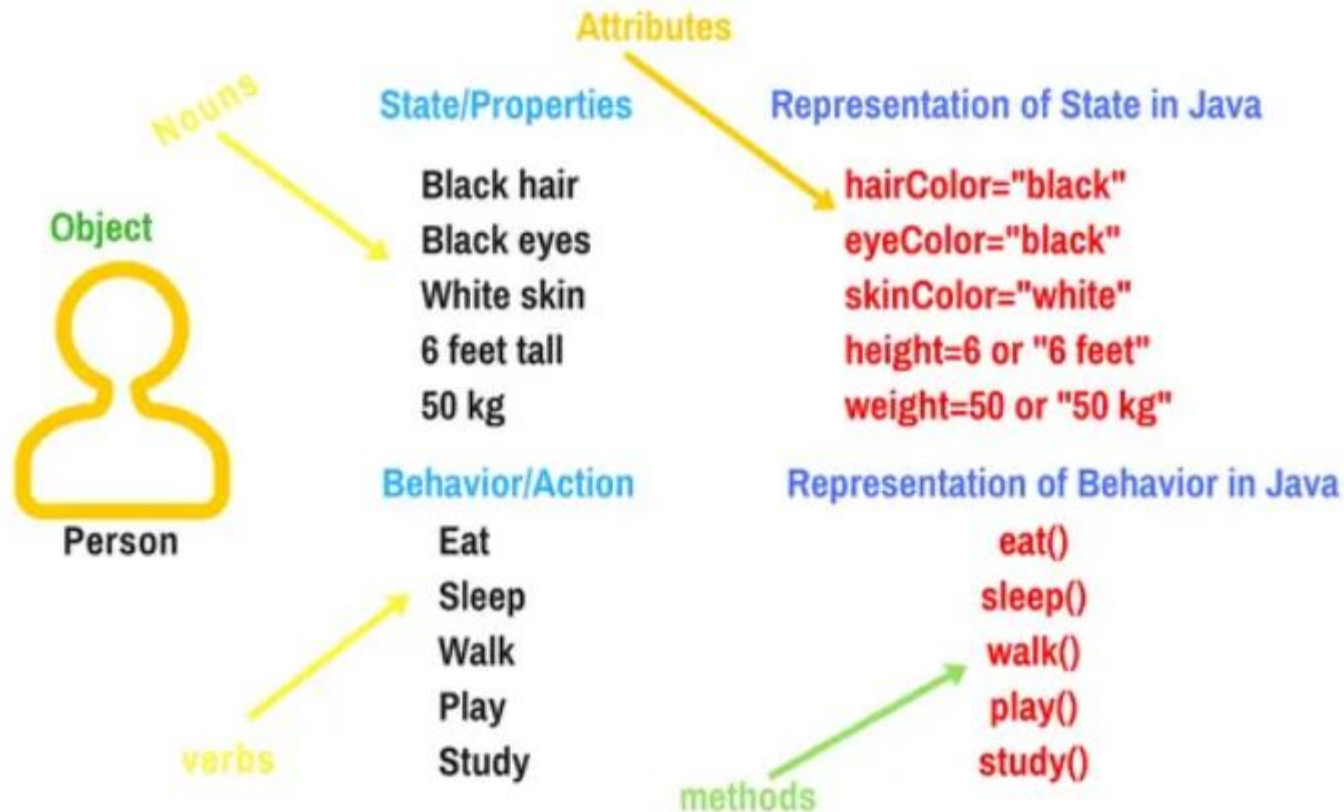
- **Object** is a variable (*instance*) of type **Class**.



CLASS VS. OBJECTS



Term	Meaning	Example
Class	Blueprint or template that defines attributes and behaviors	<pre>class clsCar { String color; int speed; void drive() {...} }</pre>
Object	A real entity created from a class	<pre>clsCar objCar = new Car();</pre>
Instance	Another word for an object; a concrete occurrence of a class	<pre>objCar</pre> is an instance of the class <pre>clsCar</pre>
Attribute	A property that describes the object	<pre>color, speed</pre>
Field	The actual variable inside the class that stores attribute values	<pre>String color; int speed;</pre>
State	The current values of an object's attributes	<pre>objCar.color = "Red"; objCar.speed = 80;</pre>
Behavior	The actions an object can perform (methods)	<pre>objCar.drive();</pre>



ENTITY → MODEL → DESIGN → CLASS → OBJECT

- We start with reality (*entity*), represent it as a model, structure it in design, implement it as a class, and finally bring it to *life* as an object.

Stage	Meaning	Example
Entity	Real-world thing	<u>Who/What:</u> A student in a university
Model	Representation of entity (attributes + behaviors)	<u>Attributes:</u> name, id, course Behaviors: registerCourse(), takeExam()
Design	Structured plan (technical blueprint)	<u>UML diagram:</u> - name: String - id: int - course: String + registerCourse() + takeExam() clsStudent - name: String - id: int - course: String + registerCourse() + takeExam() } Fields } Methods
Class	Code-level blueprint	<u>Java Class:</u> class clsStudent { private String name; private int id; private String course; public void registerCourse(String c) {...} public void takeExam() {...} }
Object	Instance in memory with state + behavior	<u>Instantiating Object:</u> clsStudent objStd = new clsStudent("Ayesha", 101); objStd .registerCourse("Object-Oriented Progr");

DATA TYPES IN JAVA

- Java classifies data types into two categories: **primitive** and **non-primitive**.

1. Primitives/System-Defined (*value-type*):

- Java provides 8 built-in primitive types (byte, short, int, long, float, double, char, and boolean).
- The programmer does not create them; they are part of the Java language specification.
- They store simple values directly (e.g., **int x = 10;**).
- Primitive data types store actual values directly in the **stack** memory.

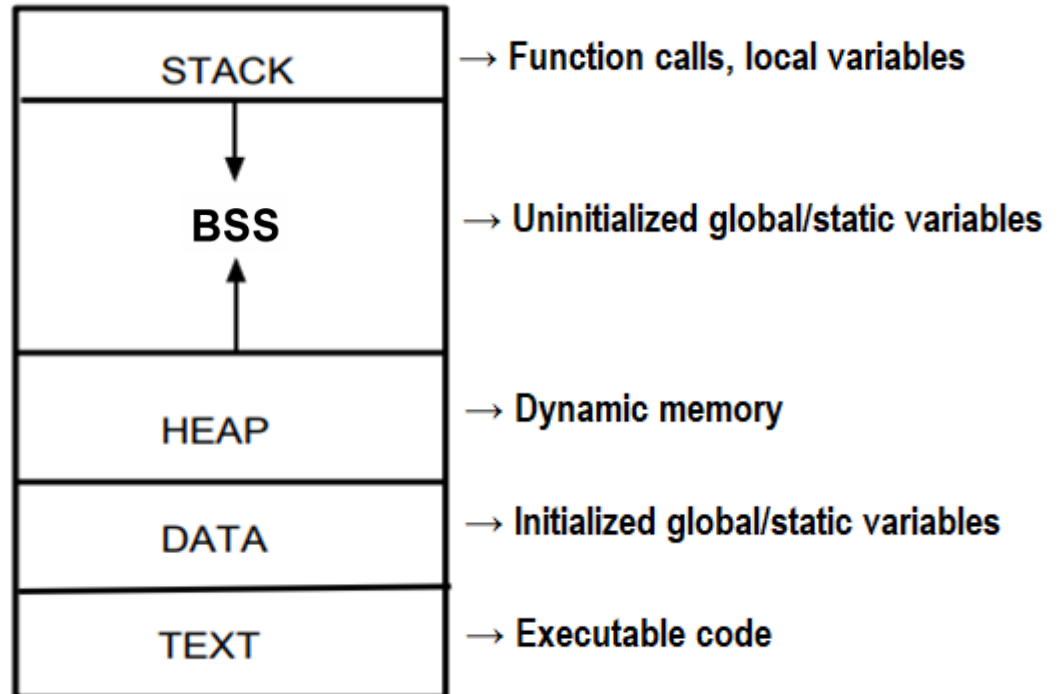
DATA TYPES IN JAVA

2. Non-Primitives Data Types (*reference type*)

- Can be system-defined (e.g., *String*, *ArrayList*) or user-defined (e.g., *Student*, *Teacher*)
- Non-primitive data types (reference types) store memory addresses (references) to objects located in the *heap* memory.
- Reference stored in *stack*, actual object in *heap memory*.

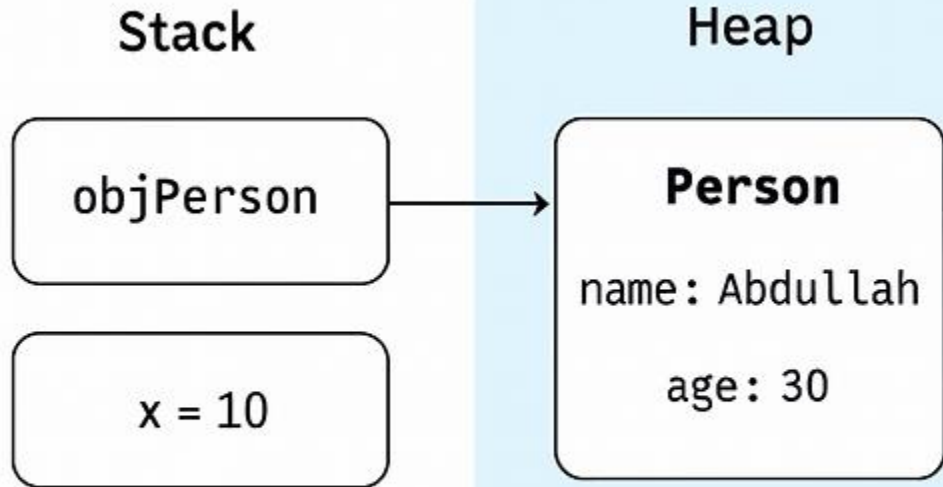
DATA TYPES IN JAVA

Memory Layout Sections



DATA TYPES IN JAVA

- `int x;`
 - `x=10;`
- } Value Type

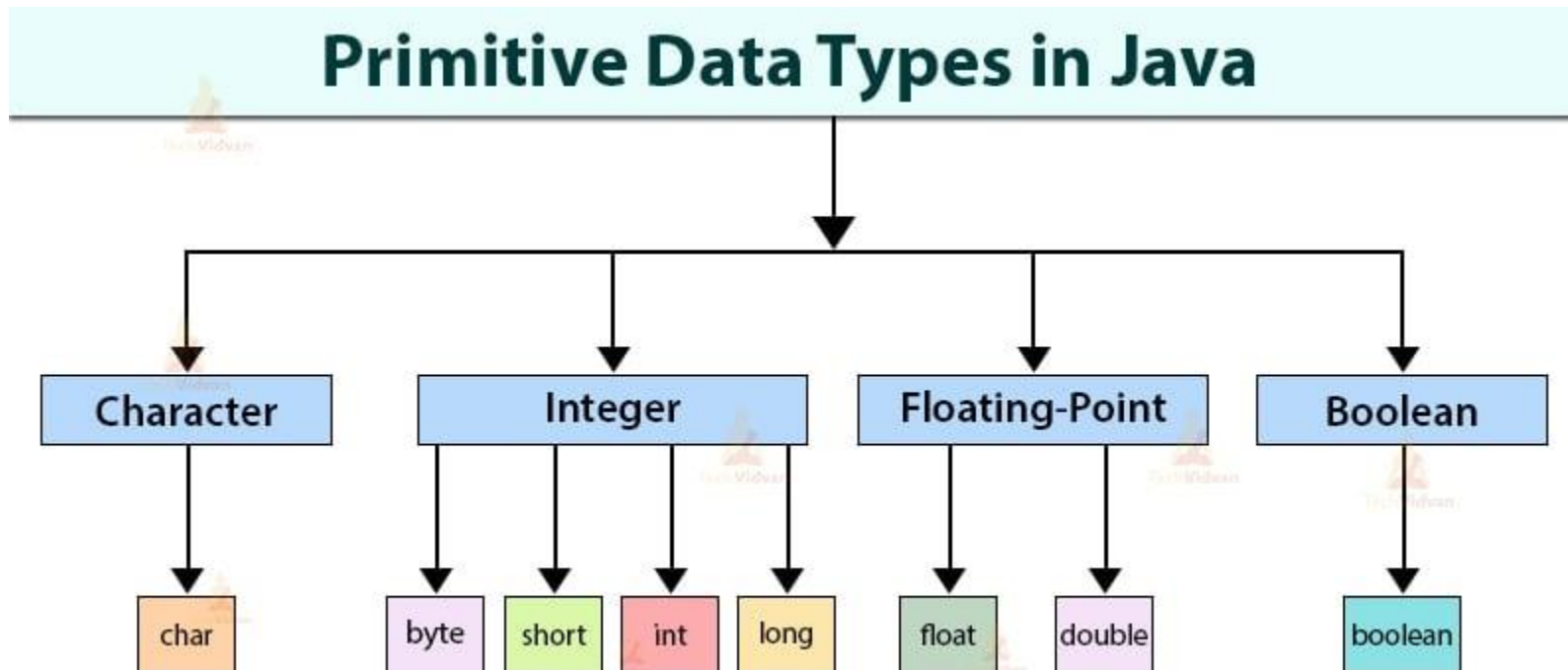


```
class Person {  
    String name;  
    int age;  
}  
class Main {  
    public static void main(String[] args) {  
        Person ObjPerson = new Person();  
        ObjPerson.name="Abdullah";  
        ObjPerson.age=30;  
    }  
}
```

DATA TYPES IN JAVA

Aspect	Value Types (Primitive Data Types)	Reference Types (Objects, Arrays, Classes)
Examples	int, double, char, boolean, byte, short, long, float	String, Array, Class, Interface, Object, Collections
Storage Location	Stored directly in stack memory	Reference stored in stack , actual object stored in heap memory
Allocation Type	Static allocation (size known at compile time)	Dynamic allocation (objects created at runtime using new)
Default Values	Numeric: 0 , Boolean: false , Char: ' \u0000 ', etc.	null
Lifetime	Tied to the scope of the variable (stack frame)	Tied to the lifetime of the object in heap; garbage collected when no references exist
Access Speed	Very fast (direct value access)	Slower (indirect access via reference pointer)
Compile-Time vs Runtime	Size and behavior determined at compile time	Actual memory allocated at runtime
Pass by	Passed by value (copy of actual data)	Passed by reference value (copy of reference, pointing to same object)

DATA TYPES IN JAVA



DATA TYPES IN JAVA

1. char

- A character is used to store a 'single' character. A single quote must surround a character. The valid Character type is **char**. In Java, a character is not an 8-bit quantity; instead, it is represented by a 16-bit Unicode.
- **Syntax:**
 - `char myChar = 'A' ;`
- **Size:**
 - 2 bytes(16 bits)
- **Values:**
 - A single character representing a digit, letter, or symbol.
- **Default Value:**
 - `'\u0000'` (0)
- **Range:**
 - `'\u0000'` (0) to `'\uffff'` (65535)

2. byte

- The byte data type is mostly used in large arrays that need memory savings. It saves memory as it is 4 times smaller than an int (integer). Sometimes, it can also be used in place of the “int” data type when the value is very small.
- The byte data type is an 8-bit signed 2's complement integer.
- **Syntax:**
 - `byte myByte1 = -100;`
 - `byte myByte2 = 25;`
- **Size:**
 - 1 byte(8 bits)
- **Values:**
 - Positive or negative whole numbers.
- **Default Value:**
 - 0
- **Range:**
 - -128 to 127

3. short

- Similar to the byte data type, a short data type is also used to save memory in large arrays. The short data type is a 16-bit signed 2's complement integer. It is half of an int (integer).
- **Syntax:**
 - `short myShort = 6000 ;`
- **Size:**
 - 2 bytes (16 bits)
- **Values:**
 - Positive or negative whole numbers.
- **Default Value:**
 - 0
- **Range:**
 - -32768 to 32767

3. **int**

- Generally, we prefer to use the int data type for integral values unless if there is no issue with memory. The int data type is a 32-bit signed 2's complement integer.
- **Syntax:**
 - `int myNum = 700000 ;`
- **Size:**
 - 4 bytes (32 bits)
- **Values:**
 - Positive or negative whole numbers.
- **Default Value:**
 - 0
- **Range:**
 - $-2,147,483,648$ (-2^{31}) to $2,147,483,647$ ($2^{31}-1$)

5. long

- The long data type is a 64-bit signed 2's complement integers. It is used when int data type cannot hold a value. It is 2 times larger than int(integer). We must use **L** at the end of the value.
- **Syntax:**
 - `long myLong = 110000000000L;`
- **Size:**
 - 8 bytes (64 bits)
- **Values:**
 - Positive or negative whole numbers.
- **Default Value:**
 - 0
- **Range:**
 - $-9,223,372,036,854,775,808(-2^{63})$ to $9,223,372,036,854,775,807(2^{63}-1)$

6. float

- The float data type is a single-precision 32-bit floating-point, based on the IEEE 754 format (single-precision floating-point numbers). We use a float instead of double to store values in a large array of floating-point numbers so as to save memory.
- Remember, we should always end the float type value with **f** or **F**, otherwise, the compiler considers it as a double value.
- **Syntax:**
 - `float myFloat = 256.8f ;`
- **Size:**
 - 4 bytes (32 bits)
- **Values:**
 - Real numbers up to 7 decimal digits.
- **Default Value:**
 - `0.0f`
- **Range:**
 - $1.40239846 \times 10^{-45}$ to $3.40282347 \times 10^{38}$

DATA TYPES IN JAVA

7. double

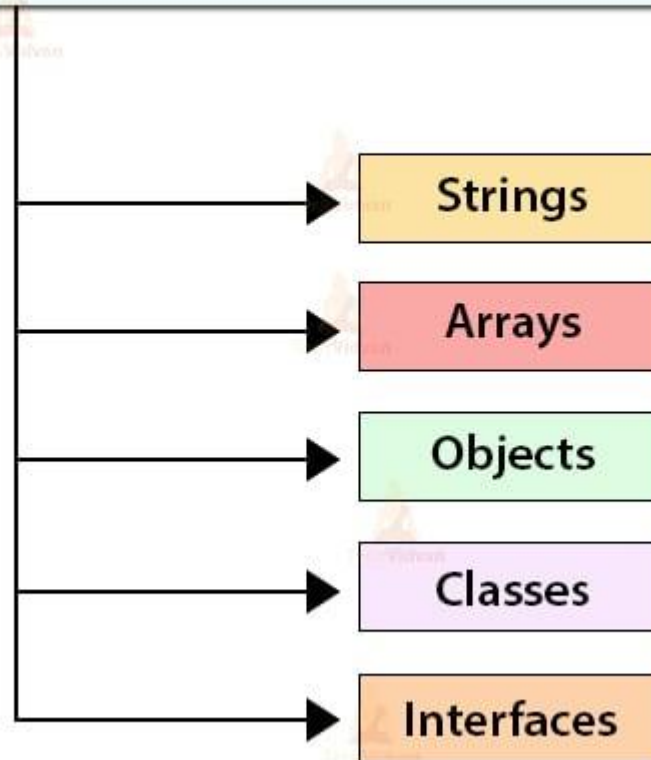
- Generally, we prefer to use the float data type for decimal values unless there is no issue with memory. As it has a precision up to 15 decimals, it is safe to use double for large calculations. We can *optionally* use the suffix **d** or **D** to end the floating type value.
- That is, both 19.67 and 19.67d are the same. The double data type is a double-precision 64-bit floating-point, based on the IEEE 754 format (double-precision floating-point numbers).
- **Syntax:**
 - double myDouble = 256.7879837 ;
- **Size:**
 - 8 bytes (64 bits)
- **Values:**
 - Real numbers up to 15 decimal digits.
- **Default Value:**
 - 0.0
- **Range:**
 - $4.9406564584124654 \times 10^{-324}$ to $1.7976931348623157 \times 10^{308}$

8. boolean Types

- A boolean data type is a 2-valued data type that is declared with the **boolean** keyword. It is capable of storing only two possible values, i.e., **true** and **false**.
- This data type is used for flag generations to check the true or false conditions. The Boolean data type stores only 1-bit information, and its size is not precisely defined.
- **Syntax:**
 - `boolean myBool = false ;`
- **Size:**
 - Machine-dependent
- **Values:**
 - true, false
- **Default Value:**
 - false
- **Range:**
 - true or false

DATA TYPES IN JAVA

Non-Primitive Data Types in Java



- **Java Strings**

- The String data type is used to store a sequence or array of characters (text). But in Java, a string is an **object** that represents an array or sequence of characters. The *java.lang.String* is a class used for creating a string object.
- String literals should be enclosed within double-quotes. The difference between a character array and a string is that in a string, a special character '\0' is present.

- **Java Strings**

- An Array in Java is a single object that can store multiple values of the same data type. Arrays are homogeneous data structures that store one or more values of a specific data type and provide indexes to access them. A particular element in an array can be accessed by its index.

- **Java Classes**

- A class is a collection of objects of the same type. It is a user-defined blueprint or prototype that defines the behavior or state of objects. Classes and objects are the basic elements of Object-Oriented Programming that represent the real-world entities.
- A class consists of a set of properties (fields or variables) or methods/operations to define the behavior of an object. We create a class using a **class** keyword.

- **Java Objects**

- It is a basic unit of Object-Oriented Programming that represents the real-world entities. An object is an instance of a class. It defines the state and behavior of real-life entities.
- **State:** It represents the attributes and properties of an object.
- **Behavior:** Methods define the behavior of an object. It also reflects the communication of one object with the other objects.
- **Identity:** It gives a unique name to an object, which allows one object to interact with other objects.

- **Java Interfaces**

- An interface is another reference type in Java.
- It is similar to a class. It can also have methods and variables, but the methods are declared implicitly as “public” and “abstract” in interfaces. However, since Java 9, we can include private methods in an interface.
- The abstract methods have only a method signature, but they can not have a method body. An interface behaves like a blueprint of a class, which specifies “what a class has to do and not how it will do”.
- In the real-world, one user defines an interface, but a different user provides its implementation. Moreover, it is finally used by some other user.

```
If(anyQuestions)
{
    askNow();
}
else
{
    thankYou();
    submitAttendance();
    endClass();
}
```